

Chapter 3 – State, Memory & Planning

Part 1 – Agent State, Memory & Persistence

1. Why Do Agents Need State?



A simple LLM interaction can be:

```
graph TD; User --> LLM; LLM --> Response;
```

But an agent may perform several steps:

```
graph TD; UserGoal[User Goal] --> LLM;
```

↓
Tool 1
↓
Result
↓
LLM
↓
Tool 2
↓
Result
↓
Final Answer

During this process, the system must remember:

what did the user ask?

which tools were called?

what results were returned?

what information has already been collected?

what should happen next?

This information is maintained through:

State

2. What is Agent State?



State is the collection of information representing the current condition of an agent workflow during execution.

Example:

```
state = {  
    "messages": [],  
    "city": "Delhi",  
    "weather": None,  
    "current_step": 1  
}
```

As execution continues, state changes.

3. State Changes During Execution

Suppose the user asks:

```
Compare Delhi and Mumbai weather.
```

Initial state:

```
{  
    "query":  
        "Compare Delhi and Mumbai weather",  
  
    "delhi_weather": None,  
  
    "mumbai_weather": None  
}
```

After Tool 1:

```
{  
  "query":  
    "Compare Delhi and Mumbai weather",  
  
  "delhi_weather": "Rainy",  
  
  "mumbai_weather": None  
}
```

After Tool 2:

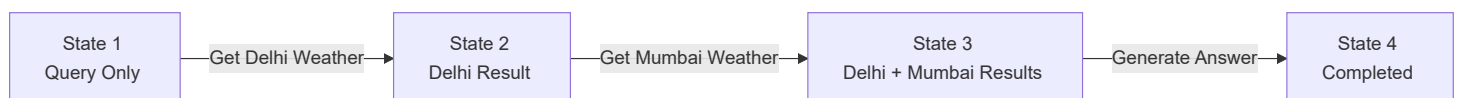
```
{  
  "query":  
    "Compare Delhi and Mumbai weather",  
  
  "delhi_weather": "Rainy",  
  
  "mumbai_weather": "Cloudy"  
}
```

Now the agent has enough information to generate the comparison.

4. State Transition

A change from one state to another is called a:

State Transition



Conceptually:

Current State
+
Action / Result
=
New State

5. What Can State Contain?

State depends on the application.

Common information includes:

Messages

User Request

Tool Results

Intermediate Results

Current Step

Execution Status

Errors

Plan

Retrieved Documents

Human Approval Status

Example:

```
state = {  
    "messages": [],  
    "tool_results": [],  
    "plan": [],  
    "current_step": 0,  
    "error": None,  
    "completed": False  
}
```

6. Message State



One of the most common forms of state is:

Message History

Example:

```
Human:  
What is the weather in Delhi?  
  
AI:  
Call get_weather("Delhi")  
  
Tool:  
31°C, Rainy  
  
AI:  
It is currently rainy in Delhi.
```

Programmatically:

```
state = {  
  "messages": [  
    human_message,  
    ai_tool_call,  
    tool_message,  
    final_ai_message  
  ]  
}
```

This allows the next model call to understand what has already happened.

7. Why State Matters in Tool Calling

Consider:

```
User  
↓  
LLM  
↓  
get_weather("Delhi")  
↓  
31°C
```

If the tool result is not preserved:

```
LLM Again  
↓  
"What was the result?"
```

The agent loses the observation.

Instead:

Tool Result



Update State



LLM Receives Updated State



Next Decision

8. State in an Agent Loop



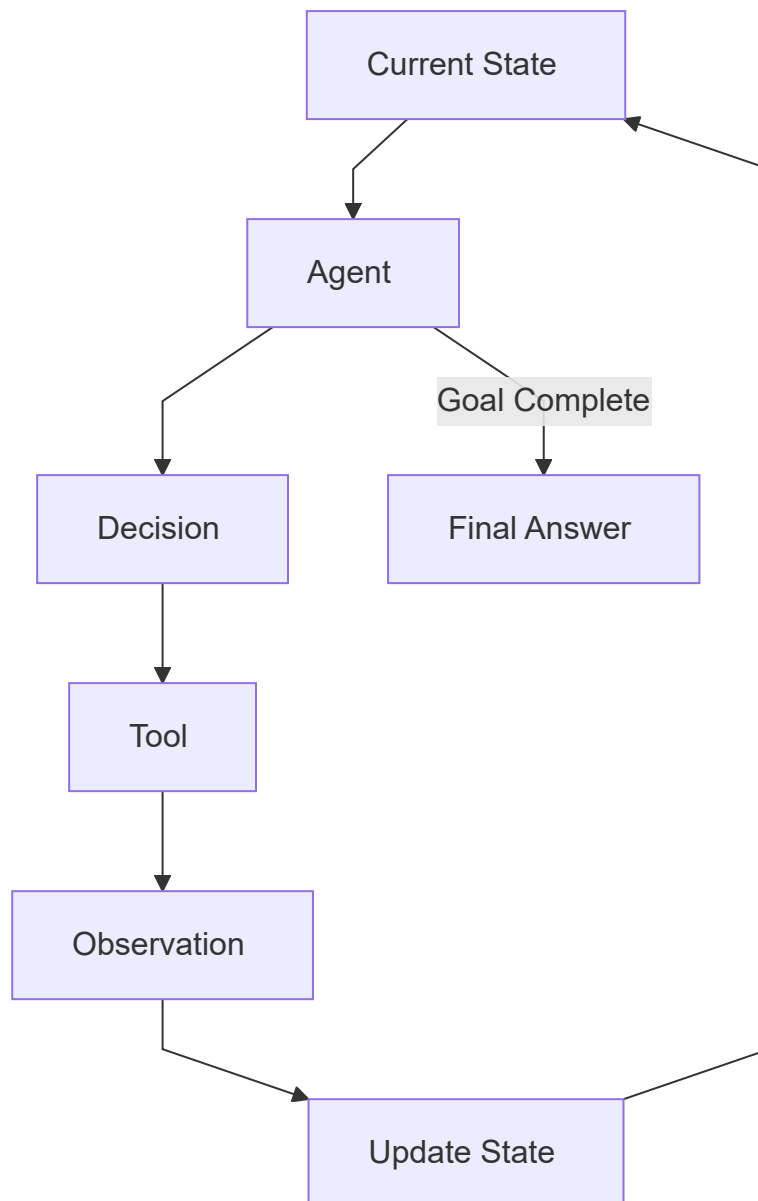
```
state = {  
    "messages": [],  
    "steps": 0  
}  
  
while True:  
  
    decision = agent(state)  
  
    result = execute(  
        decision  
    )  
  
    state = update_state(  
        state,  
        result  
    )  
  
    if goal_completed(state):  
        break
```

The loop does not operate on isolated requests.

Every iteration receives the:

Updated State

9. State Architecture



This pattern becomes the foundation of:

LangGraph

because LangGraph workflows are heavily based around explicit state transitions.

10. State vs Context

These terms are related but not identical.

State

Information maintained by the application/workflow.

Example:

```
{  
  "messages": [...],  
  "weather": {...},  
  "step": 3  
}
```

Model Context

The information actually sent to the LLM during a particular model call.

Example:

System Instructions

User Message

Recent Messages

Relevant Memory

Tool Results

Therefore:

State

→ Information maintained by system

Context

→ Information supplied to model

Not every piece of state must necessarily be placed into every LLM prompt.

II. Why Not Send Everything to the LLM?

Suppose an agent has been running for a long time.

State may contain:

500 messages

100 tool results

Large documents

Logs

Metadata

Sending everything to the LLM every time causes:

Large context

Higher cost

Higher latency

Context-window problems

Irrelevant information

Therefore:

Application State



Select Relevant Information



LLM Context

12. What is Memory?



Memory allows an agent to retain and later retrieve useful information from previous interactions or earlier events.

Example:

User:
I prefer Hindi responses.

Later:

User:
Explain this crop recommendation.

If the preference has been stored as useful memory, the application can retrieve it and provide it to the agent.

13. State vs Memory

★★★★★ *Very Important Interview Question*

A simple mental model:

STATE
=
what is happening in the current workflow?

MEMORY
=
what useful information should be retained
and retrieved later?

Example:

Current tool result:
Rainy in Delhi

→ State

Example:

User prefers Celsius

→ Could be Memory

14. State vs Memory Table

Feature	State	Memory
Main purpose	Track current execution	Retain useful information
Lifetime	Often workflow/thread	Can persist longer
Tool results	Common	Sometimes
Current step	Yes	Usually no
User preferences	Sometimes	Common
Persistence required	Not always	Often
Used across sessions	Not necessarily	Can be

The exact distinction depends on the framework and architecture.

15. Short-Term Memory



Short-term memory usually refers to information relevant to the current conversation or active interaction.

Example:

User:
Compare wheat and rice.

Assistant:
Which region?

User:
Uttar Pradesh.

The agent needs to remember:

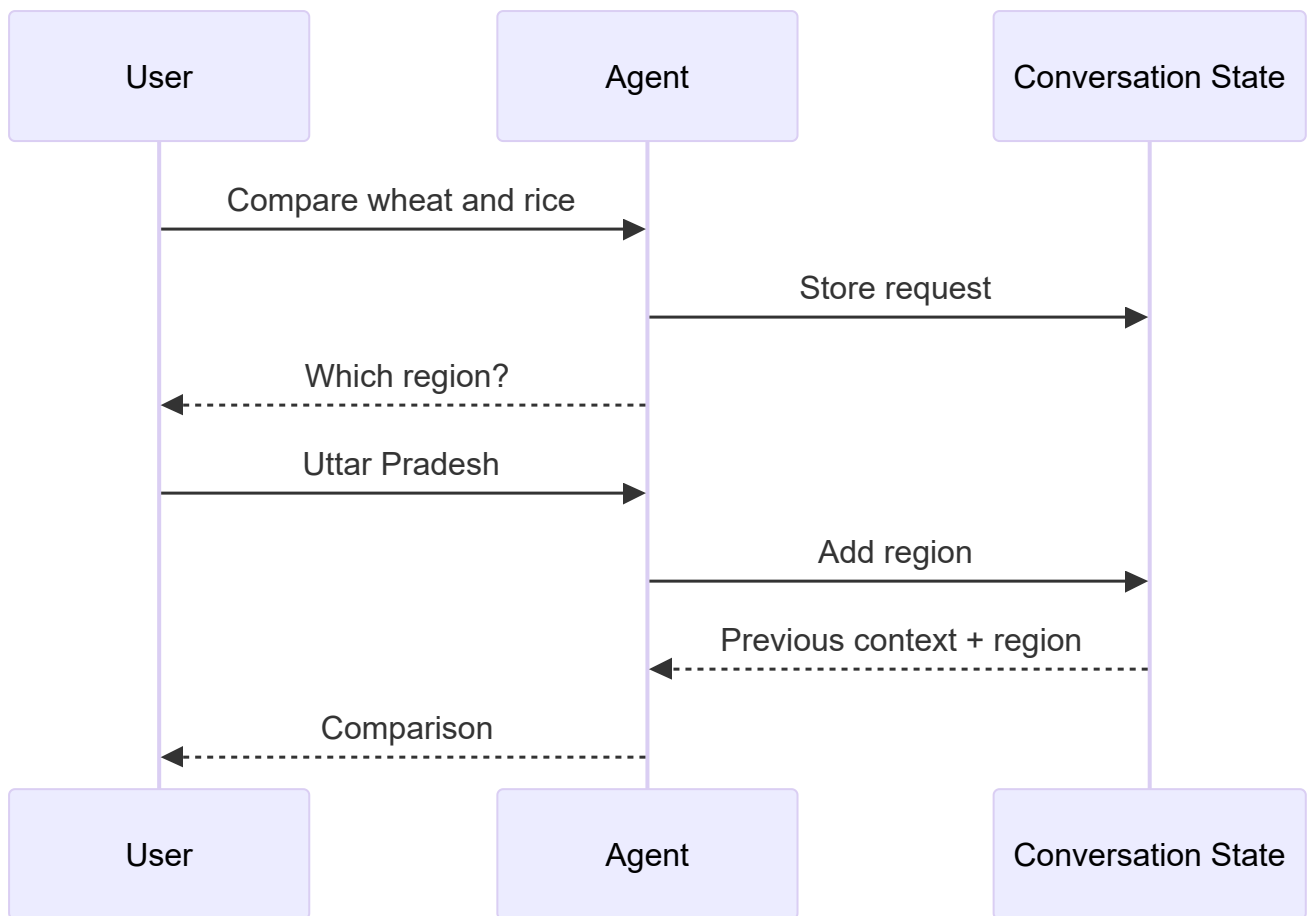
Original task
+
Region = Uttar Pradesh

Otherwise:

"Uttar Pradesh"

alone has little meaning.

16. Short-Term Memory Flow



17. Long-Term Memory



Long-term memory refers to information retained beyond the immediate interaction.

Examples:

User preferences

Important user facts

Past decisions

Previous task outcomes

Application-specific knowledge

Possible storage:

SQL Database

MongoDB

Key-Value Store

Vector Database

Other Persistent Storage

18. Short-Term vs Long-Term Memory

SHORT-TERM MEMORY

Current conversation

Current task

Recent messages

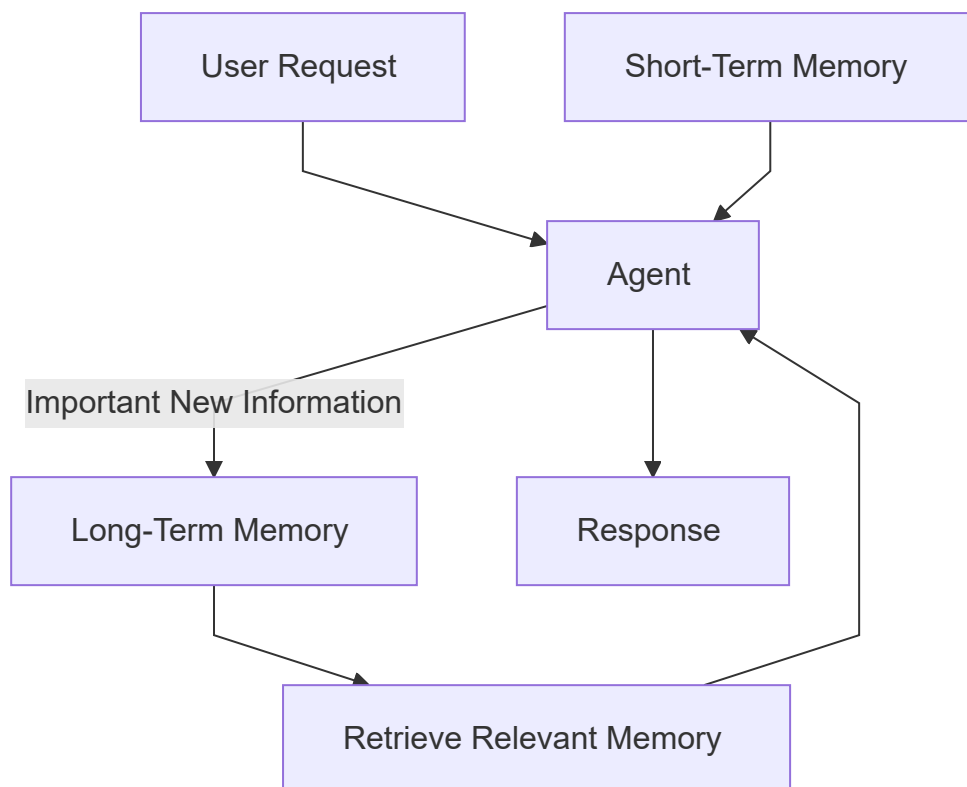
Temporary information

versus:

LONG-TERM MEMORY

- Persistent preferences
- Useful past information
- Previous decisions
- Cross-session information

Architecture:



19. Persistent Memory

Memory becomes persistent when it is stored outside temporary process memory.

For example:

```
memory = {}
```

disappears when the application restarts.

But:

```
Agent
  ↓
Database
  ↓
Stored Memory
```

can survive application restarts.

20. Example Persistent Memory

Database record:

```
{
  "user_id": "user_123",

  "preferences": {
    "language": "Hindi",
    "temperature_unit": "celsius"
  }
}
```

Later:

```
graph TD; A[User Request] --> B[Get User ID from Authenticated Session]; B --> C[Retrieve Preferences]; C --> D[Add Relevant Preferences to Context]; D --> E[LLM];
```

21. Memory Should Be Selective



Do not store everything forever.

Bad architecture:

```
graph TD; A[Every Message] --> B[Every Tool Result]; B --> C[Every Temporary Detail]; C --> D[Every Error]; D --> E[Permanent Memory];
```

Problems:

Storage growth

Irrelevant memories

Privacy concerns

Poor retrieval

Conflicting information

Instead determine:

Is this information useful later?

Is it appropriate to store?

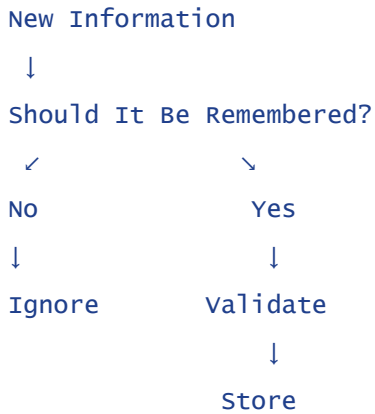
How long should it remain?

Can it become outdated?

22. Memory Write

The process of storing information can be thought of as:

Memory Write



Example:

```
User:
Always show temperature in celsius.
```

Potential memory:

```
{
  "temperature_unit": "celsius"
}
```

23. Memory Read

Before responding:

User Request



Determine Relevant Memory



Retrieve Memory



Add Relevant Information to Context

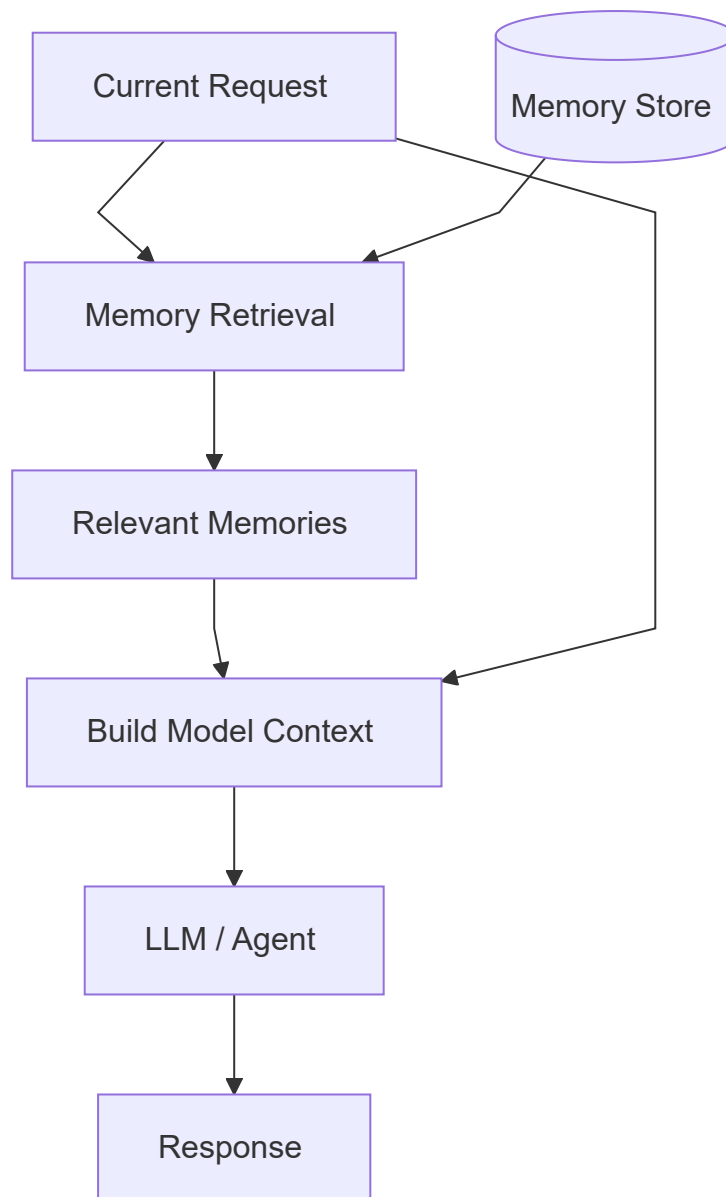


Agent

This is:

Memory Retrieval

24. Memory Retrieval Architecture



25. Semantic Memory Retrieval

Suppose thousands of memories exist.

We may not want exact-key lookup only.

Example query:

How should I format the weather?

Stored memory:

User prefers temperatures in Celsius.

These sentences are semantically related.

Embeddings and vector search can help retrieve relevant memories.

Conceptually:

```
graph TD; A[Current Query] --> B[Embedding]; B --> C[Vector Search]; C --> D[Relevant Memories]; D --> E[LLM Context]
```

26. Vector Memory

Since you already studied embeddings and vector databases, the idea is straightforward:

Memory



Embedding Model



Vector



Vector Database

Later:

Current Request



Embedding



Similarity Search



Relevant Memory

This is conceptually similar to retrieval in RAG.

27. Memory vs RAG

They can look similar because both may use retrieval.

RAG

Usually retrieves:

External knowledge

Documents

Knowledge base information

Example:

Crop cultivation guide

Agent Memory

Usually retrieves information related to:

Previous interactions

User preferences

Past decisions

Agent experience/state history

Example:

User prefers organic fertilizer recommendations.

So:

RAG

→ Retrieve knowledge

Memory

→ Retrieve relevant past information

The underlying retrieval technology may be similar.

28. Different Memory Storage Strategies

Memory does not always require a vector database.

Use the simplest appropriate storage.

Structured Database

Best for:

User preferences

Settings

Known attributes

IDs

Structured facts

Example:

user_id | language | unit

Key-Value Store

Useful for:

Simple preferences

Session information

Cached state

Vector Store

Useful for:

Semantic retrieval

Unstructured memories

Past conversation snippets

29. Don't Use Vector Databases for Everything



Suppose you need:

`preferred_language = "Hindi"`

You do not need:

Embedding

↓

Vector Database

↓

Similarity Search

A normal database field is better.

Use:

Structured information

→ Structured database

Semantic unstructured information

→ Vector retrieval

30. Conversation History vs Memory

Another important distinction.

Conversation History

User message

Assistant response

User message

Tool message

Assistant response

It records the conversation.

Memory

Memory is information intentionally retained or retrieved because it may be useful.

For example, from:

User:
I prefer concise answers because I use
these notes for interview revision.

Conversation history contains the whole sentence.

Memory might store:

Response preference:
concise interview-oriented explanations

So:

History
→ Raw interaction record

Memory
→ Useful retained information

31. Memory Extraction

A system may extract useful memory from conversation.

Conceptually:

```
Conversation
↓
Memory Extraction
↓
Candidate Information
↓
Validate / Filter
↓
Store
```

Example:

```
Conversation:

User:
I prefer celsius.
```

Extracted memory:

```
{
  "type": "preference",
  "key": "temperature_unit",
  "value": "celsius"
}
```

32. Memory Updating



Memories can change.

Suppose memory contains:

Preferred language:
English

Later user says:

From now on, use Hindi.

The system should not keep blindly using the old preference.

Instead:

Existing Memory
+
New Information
↓
Conflict?
↓
Update / Replace

33. Memory Can Become Stale

Examples:

Current city

Current job

Temporary project

Preferred tool

Travel plans

can change.

Therefore useful memory systems may need:

Timestamps

Update rules

Expiration

Conflict handling

34. Memory Metadata

A memory can contain metadata.

Example:

```
{  
  "content":  
    "User prefers Celsius",  
  
  "type":  
    "preference",  
  
  "created_at":  
    "2026-08-01",  
  
  "updated_at":  
    "2026-08-04"  
}
```

Metadata helps manage memory lifecycle.

35. Memory Scope

Memory should have a clear scope.

Examples:

User-level memory

Conversation-level memory

Project-level memory

Agent-level memory

For example:

User Preference

→ User-level

Current research task

→ Conversation/task-level

Project architecture decision

→ Project-level

36. Checkpointing



Another concept closely related to state is:

Checkpointing

A checkpoint stores the workflow state at a particular point so execution can potentially continue later.

Example:

```
graph TD; A[Agent Starts] --> B[Step 1]; B --> C[Checkpoint]; C --> D[Step 2]; D --> E[Checkpoint]; E --> F[Human Approval Required]; F --> G[Pause];
```

Later:

```
graph TD; H[Resume] --> I[Load Checkpoint]; I --> J[Continue from Previous State];
```

37. Why Checkpointing Matters

Without checkpointing:

Long workflow



Application Stops



Everything Lost



Restart From Beginning

With checkpointing:

Long workflow



Save State



Application Stops



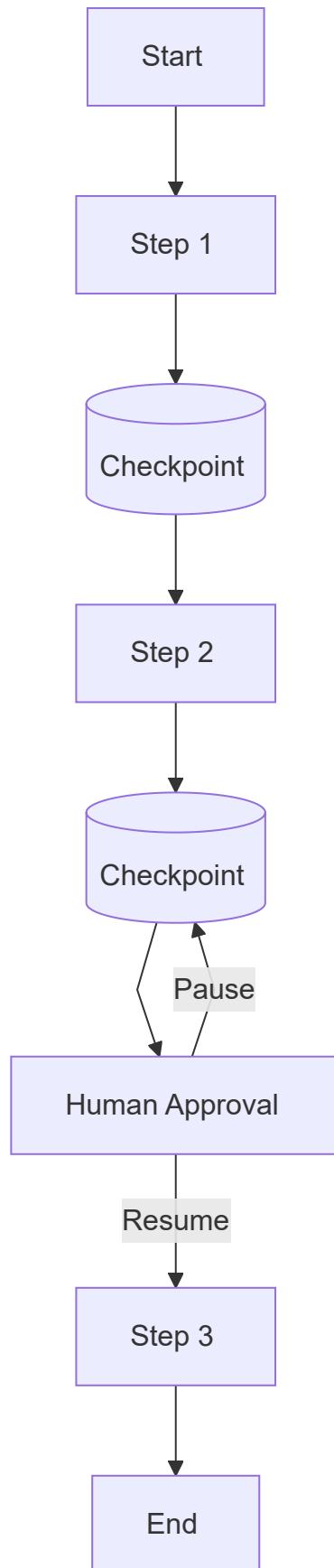
Restore State



Continue



38. Checkpoint Architecture



Checkpointing becomes especially important when we study:

LangGraph

Persistence

Human-in-the-loop

39. State Persistence vs Memory

Do not confuse these.

State Persistence

Save the current workflow so it can continue.

Example:

Agent is currently waiting for approval.

Memory

Store useful information for future use.

Example:

User prefers Celsius.

So:

Persistent State

→ Resume workflow

Memory

→ Improve future interactions

40. Thread / Conversation Identity

To restore the correct state, the application needs an identifier.

Conceptually:

thread_id

Example:

```
config = {  
  "thread_id":  
    "conversation_123"  
}
```

Then:

```
thread_123  
↓  
Checkpoint Store  
↓  
Correct Saved State
```

Different conversations can maintain separate states.

41. Basic State Implementation

Let's implement a simple stateful agent structure.

```
class AgentState:

    def __init__(self):

        self.messages = []

        self.tool_results = []

        self.steps = 0

        self.completed = False
```

Create state:

```
state = AgentState()
```

Add a message:

```
state.messages.append(
    "Find Delhi weather"
)
```

Add tool result:

```
state.tool_results.append(  
    {  
        "tool": "get_weather",  
        "result": "Rainy"  
    }  
)
```

42. Dictionary-Based State

Often we can simply use:

```
state = {  
    "messages": [],  
    "tool_results": [],  
    "steps": 0,  
    "completed": False  
}
```

Update:

```
state["steps"] += 1
```

Store observation:

```
state["tool_results"].append(  
    {  
        "tool": "get_weather",  
        "result": "Rainy"  
    }  
)
```

43. Simple Persistent Memory Implementation

For learning purposes:

```
memory_store = {}
```

Save:

```
def save_preference(
    user_id,
    key,
    value
):

    if user_id not in memory_store:
        memory_store[user_id] = {}

    memory_store[user_id][key] = value
```

Use:

```
save_preference(
    "user_123",
    "temperature_unit",
    "celsius"
)
```

44. Retrieve Memory

```
def get_preferences(user_id):  
  
    return memory_store.get(  
        user_id,  
        {}  
    )
```

Then:

```
preferences = get_preferences(  
    "user_123"  
)
```

Result:

```
{  
    "temperature_unit":  
        "celsius"  
}
```

In production this would typically be backed by persistent storage rather than a Python dictionary.

45. Combining State and Memory

Suppose:

Memory:

User prefers Celsius

Current State:

Delhi temperature = 31°C

The agent can combine both:

Current Request

+

Current State

+

Relevant Memory

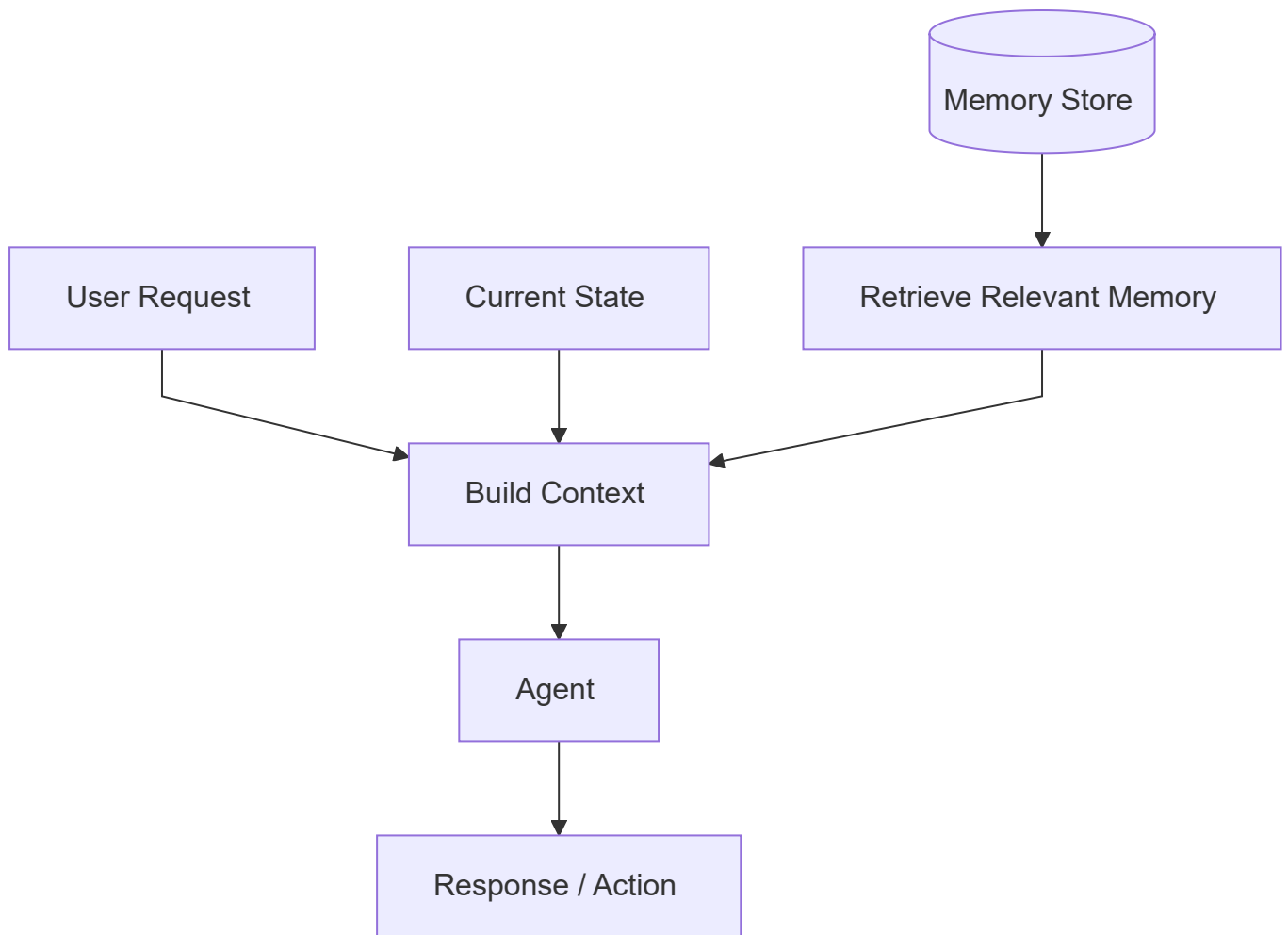
↓

LLM

↓

Response

Diagram:



46. Practical Example

User asks:

what's the weather in my preferred city?

Memory:

```
{  
  "preferred_city": "DeLhi",  
  "temperature_unit": "ceLsius"  
}
```

Agent retrieves memory:

```
preferred_city  
→ Delhi  
  
temperature_unit  
→ Celsius
```

Then:

```
get_weather("Delhi")
```

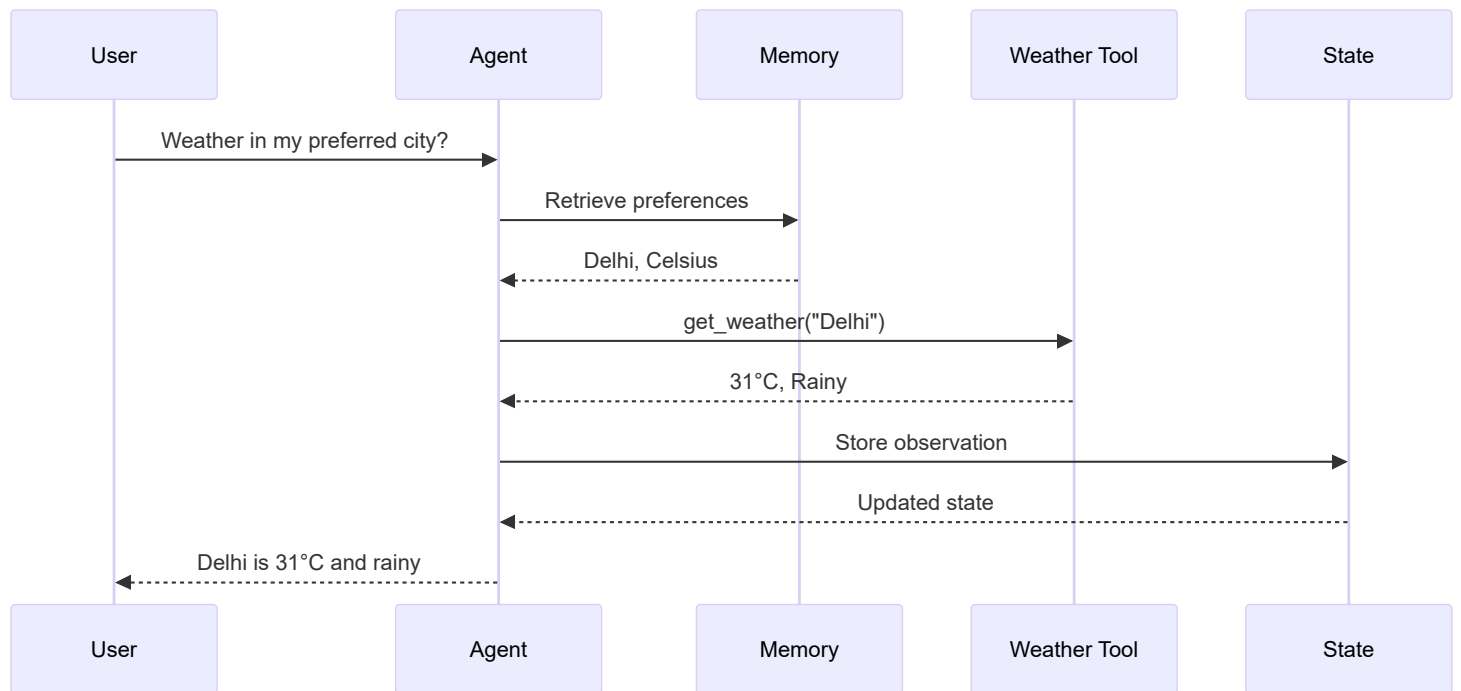
Observation:

```
31°C, Rainy
```

Final answer:

```
Delhi is currently 31°C and rainy.
```

47. Complete Flow



48. State + Memory + Tool Calling

We can now connect the first three major concepts:

STATE

→ Tracks current execution

MEMORY

→ Retrieves useful past information

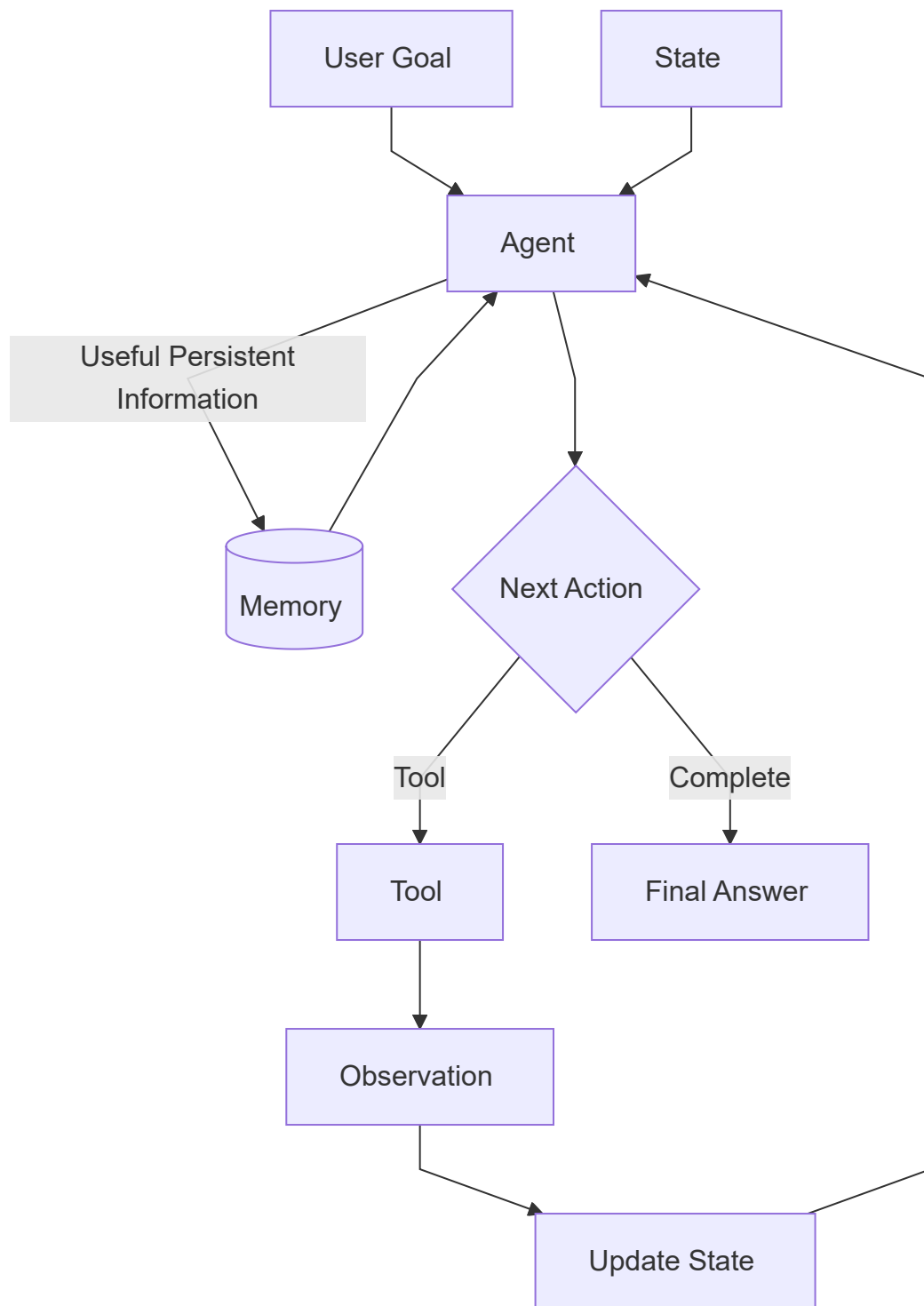
TOOLS

→ Interact with external systems

LLM

→ Makes flexible decisions

Together:



49. Common Mistakes

Mistake 1 – Treating State and Memory as Identical

Remember:

```
State
→ Current workflow

Memory
→ Retained useful information
```

Mistake 2 – Storing Everything

Memory should be selective.

Mistake 3 – Sending Entire State to LLM

Only provide relevant context.

Mistake 4 – Using Vector DB for Every Memory

Structured data should usually stay structured.

Mistake 5 – Never Updating Memory

Preferences and facts can change.

Mistake 6 – No Memory Scope

Know whether information belongs to:

User

Conversation

Project

Agent

Mistake 7 – Confusing Checkpointing With Memory

Checkpoint

→ Resume execution

Memory

→ Retain useful information

50. Important Interview Questions

Q1. What is state in an AI agent?



Answer:

State is the information representing the current condition of an agent workflow, such as messages, intermediate results, tool observations, current steps and execution status.

Q2. Why is state important?

Answer:

State allows an agent to preserve information between steps so that later decisions can use the results of previous actions.

Q3. What is a state transition?

Answer:

A state transition is the change from one workflow state to another after an action, observation or other event updates the system.

Q4. State vs model context?

Answer:

State is information maintained by the application or workflow, while model context is the subset of information supplied to the LLM for a particular model call.

Q5. What is agent memory? ★ ★ ★ ★ ★

Answer:

Agent memory is a mechanism for retaining and retrieving useful information from previous interactions or events so it can influence future behavior.

Q6. State vs memory? ★ ★ ★ ★ ★

Answer:

State primarily tracks the current workflow execution, while memory retains useful information that may be retrieved later, potentially across different interactions or sessions.

Q7. Short-term vs long-term memory?

Answer:

Short-term memory generally maintains information relevant to the active conversation or task, while long-term memory persists useful information beyond the immediate interaction.

Q8. Does long-term memory require a vector database? ★ ★ ★ ★ ★

Answer:

No. Structured information such as user preferences can be stored in a normal database. Vector databases are useful when semantic retrieval over unstructured memories is required.

Q9. Memory vs RAG? ★ ★ ★ ★ ★

Answer:

RAG usually retrieves external knowledge from documents or knowledge bases, while memory retrieves useful information from previous interactions, preferences or agent history. Both can use similar retrieval techniques.

Q10. What is checkpointing? ★ ★ ★ ★ ★

Answer:

Checkpointing saves workflow state at particular points so that execution can be restored or resumed later instead of restarting from the beginning.

Q11. Checkpointing vs memory?

Answer:

Checkpointing primarily preserves execution state for workflow recovery or continuation, while memory stores useful information intended to influence future interactions or decisions.

Q12. Why shouldn't an agent store everything as memory?

Answer:

Storing everything increases noise, storage requirements, privacy risks and retrieval errors. Memory should be selective and contain information that is useful and appropriate for future interactions.

Q13. What is semantic memory retrieval?

Answer:

Semantic memory retrieval uses meaning-based similarity, often through embeddings and vector search, to retrieve memories relevant to the current request even when the wording is different.

Q14. What is memory scope?

Answer:

Memory scope defines where a memory applies, such as to a particular user, conversation, project or agent.

51. Final Mental Model

Remember:

STATE

"what is happening now?"

MEMORY

"what useful information from before
should I remember?"

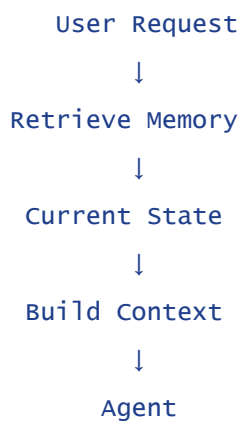
CHECKPOINT

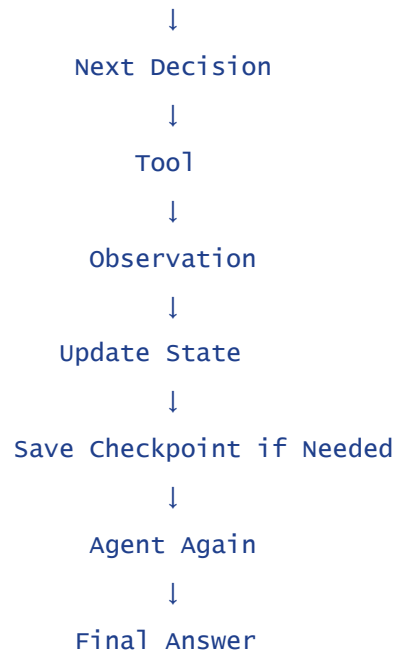
"where was the workflow so I can
resume it?"

CONTEXT

"what information should the LLM
see right now?"

Complete architecture:





The key distinction:

State

→ Current execution information

Memory

→ Retained information

Context

→ Information sent to LLM

Checkpoint

→ Saved workflow state

State gives an agent continuity within a workflow, while memory gives it continuity across useful past interactions. Together they allow agents to move beyond isolated prompt-response behavior into stateful, context-aware systems.

Chapter 3 – State, Memory & Planning

Part 2 – Planning, Task Decomposition, Reflection & Retry

1. Why Do Agents Need Planning?



Some tasks can be solved with one action.

Example:

```
User:
What is the weather in Delhi?

Agent:
Call weather Tool
↓
Return Answer
```

But complex tasks may require several actions.

Example:

User:

Compare the current weather of Delhi and Mumbai and recommend which city is better for an outdoor event.

The agent needs to:

1. Get Delhi weather
2. Get Mumbai weather
3. Compare both results
4. Evaluate outdoor suitability
5. Generate recommendation

This introduces:

Planning

2. What is Planning?



Planning is the process of determining the steps or actions required to achieve a goal.

Basic flow:

Goal



Understand Goal



Break Into Tasks



Determine Actions



Execute



Observe Results



Adjust if Required



Complete Goal

3. Goal vs Plan vs Action

These three concepts should be clearly separated.

Goal

What needs to be achieved?

Find the better city for an outdoor event.

Plan

What steps are required?

1. Get Delhi weather
2. Get Mumbai weather
3. Compare conditions
4. Recommend city

Action

What operation should happen now?

```
get_weather("Delhi")
```

So:

```
Goal
  ↓
Plan
  ↓
Tasks
  ↓
Actions
  ↓
Observations
  ↓
Result
```

4. Task Decomposition



Task decomposition means breaking a complex goal into smaller manageable tasks.

Example:

Goal:

Research a company and prepare
a short report.

Decomposition:

Task 1

→ Find company overview

Task 2

→ Find products/services

Task 3

→ Find recent developments

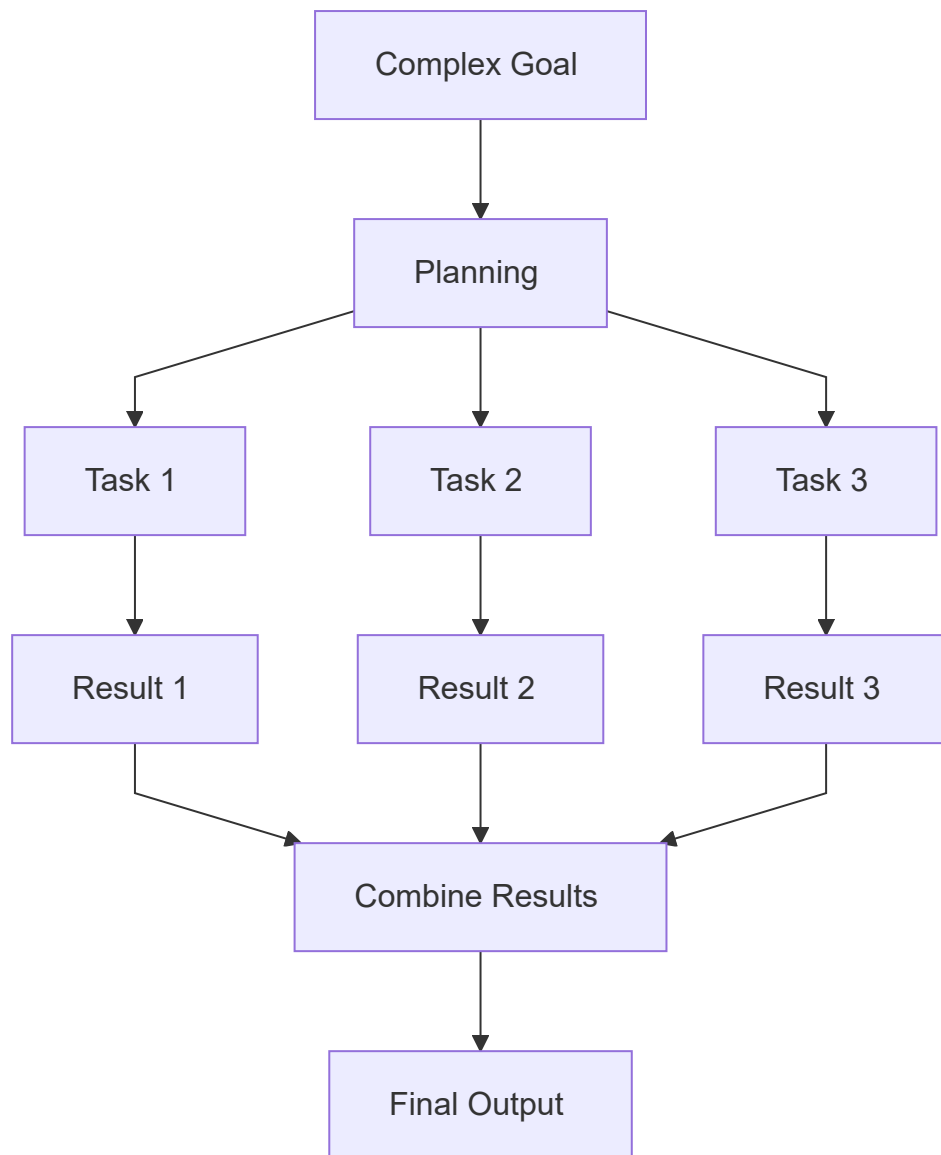
Task 4

→ Organize information

Task 5

→ Generate report

5. Task Decomposition Architecture



6. Why Task Decomposition Helps

Breaking a task into smaller steps can improve:

Clarity

Tool selection

Execution control

Error handling

Observability

Retry behavior

Instead of asking the agent to:

"Do everything."

the system can reason about manageable subtasks.

7. Fixed Planning vs Dynamic Planning



Fixed Plan

The workflow is known beforehand.

Step 1



Step 2



Step 3



Output

Example:

Upload File



Extract Text



Summarize



Save Summary

This may be better implemented as a normal deterministic workflow.

Dynamic Planning

The next action depends on observations.

Goal



Action



Observation



Decide Next Action



Observation



...

Example:

```
graph TD; A[Search product] --> B[Product unavailable]; B --> C[Search alternative]; C --> D[Alternative too expensive]; D --> E[Search another option];
```

This is more agentic.

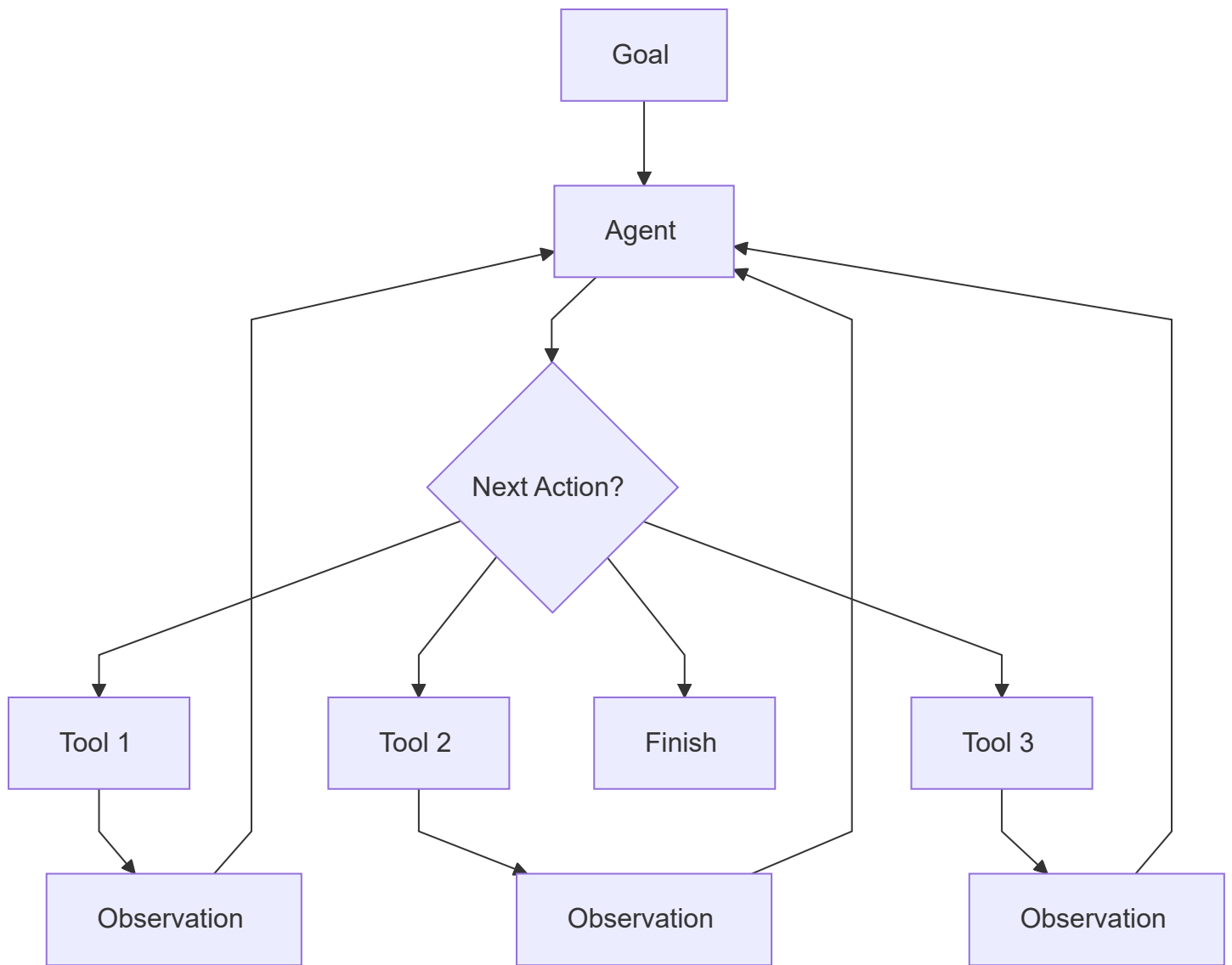
8. Static Workflow vs Dynamic Agent



Static workflow:

Path known beforehand.

Agentic workflow:



The path can change during execution.

9. Planning Strategies

At a high level, agent planning can happen in different ways.

The most useful patterns to understand are:

1. Step-by-Step / Reactive Planning
2. Plan-and-Execute
3. Replanning
4. Reflection / Self-Correction

We do not need to go deeply into research-level planning algorithms.

10. Reactive Planning

In a reactive approach, the agent decides only the:

Next Best Action

rather than generating a complete plan beforehand.

Flow:

```
graph TD; A[Current State] --> B[Decide Next Action]; B --> C[Execute]; C --> D[Observation]; D --> E[Updated State]; E --> F[Decide Next Action];
```

This is closely related to the agent loop we already studied.

11. Reactive Agent Example

User:

Find the current weather in Delhi
and tell me if I should carry an umbrella.

Agent:

Current State:
Need current weather.

↓

Action:
get_weather("Delhi")

↓

Observation:
Rain expected.

↓

New Decision:
Enough information available.

↓

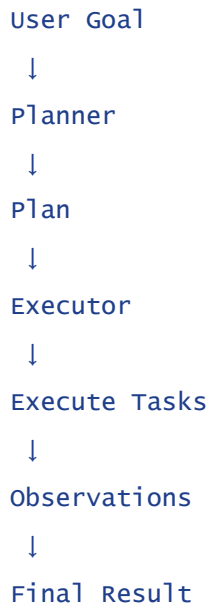
Final Answer:
Carry an umbrella.

The agent did not need to create a large plan beforehand.

12. Plan-and-Execute Pattern



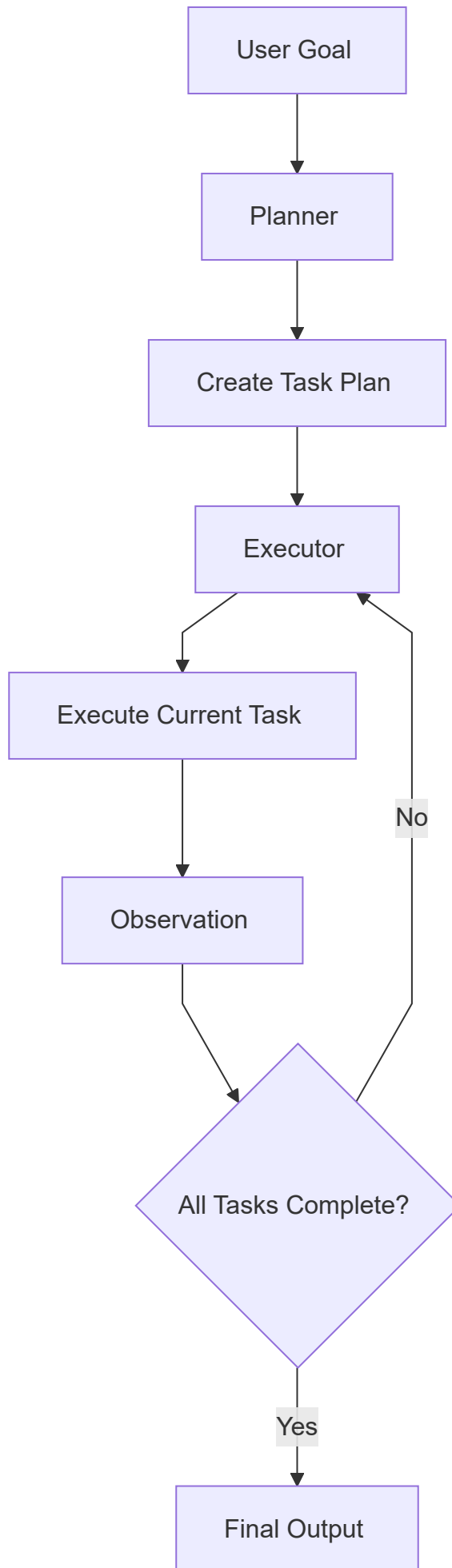
For larger tasks, an agent can first create a plan.



```
graph TD; A[User Goal] --> B[Planner]; B --> C[Plan]; C --> D[Executor]; D --> E[Execute Tasks]; E --> F[Observations]; F --> G[Final Result];
```

User Goal
↓
Planner
↓
Plan
↓
Executor
↓
Execute Tasks
↓
Observations
↓
Final Result

Diagram:



13. Planner

The planner determines:

- what needs to be done?
- which tasks are required?
- what order should they follow?
- which tasks depend on others?

Example:

- Goal:
Prepare a weather comparison report.
 - Plan:
 1. Get Delhi weather
 2. Get Mumbai weather
 3. Compare temperatures
 4. Compare rain conditions
 5. Generate recommendation
-

14. Executor

The executor performs the tasks.

Example:

Planner says:

Task 1:

Get Delhi weather.

Executor:

```
get_weather("Delhi")
```

Observation:

31°C, Rainy

Then the executor proceeds to the next task.

15. Planner vs Executor



Planner

→ Determines WHAT should be done

Executor

→ Performs the required action

They do not necessarily need to be separate AI models.

A single agent can perform both roles.

16. Plan Representation

A plan can be stored in state.

Example:

```
state = {  
  
    "goal":  
        "Compare Delhi and Mumbai weather",  
  
    "plan": [  
        "Get Delhi weather",  
        "Get Mumbai weather",  
        "Compare results",  
        "Generate recommendation"  
    ],  
  
    "current_step": 0,  
  
    "results": {}  
}
```

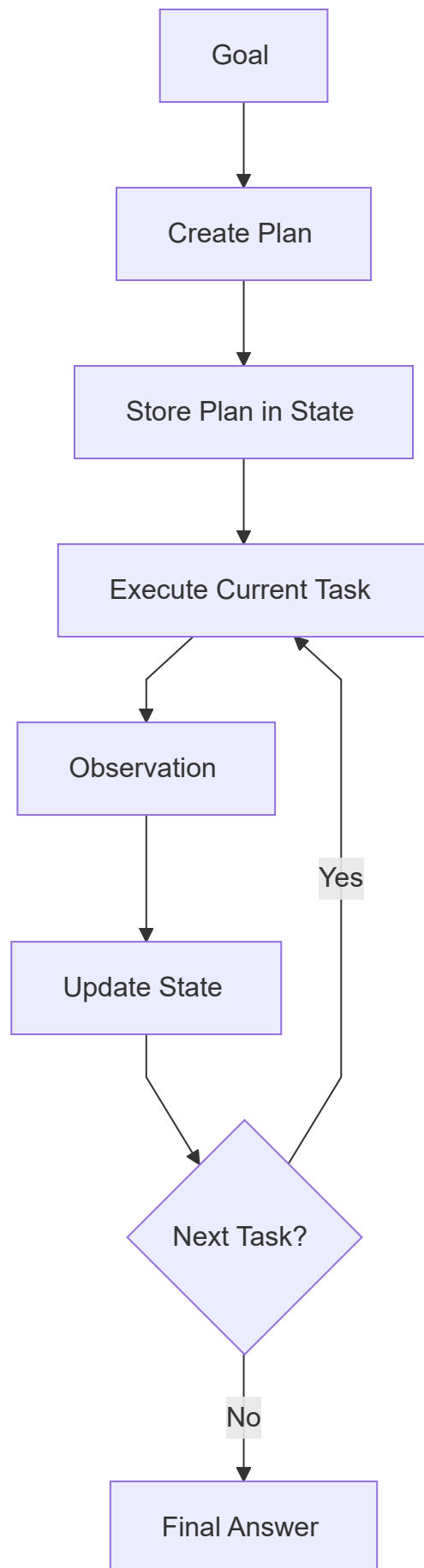
After one task:

```
state["current_step"] = 1  
  
state["results"]["delhi"] = {  
    "temperature": 31,  
    "condition": "rainy"  
}
```

17. Planning + State



Planning and state work together.



State tracks:

Plan

Current Task

Completed Tasks

Observations

Errors

18. Task Dependencies

Not all tasks can execute in any order.

Example:

Task 1:

Get current Bitcoin price.

Task 2:

Calculate 10% of Bitcoin price.

Task 2 requires Task 1.

Task 1

↓

Result

↓

Task 2

This is a:

Dependency

19. Independent Tasks

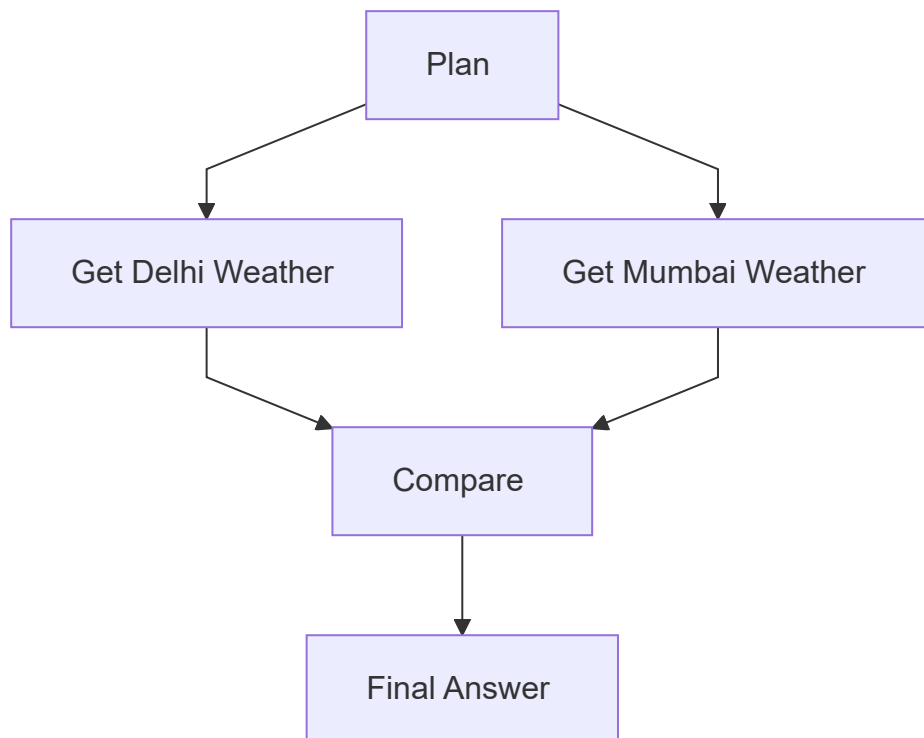
Some tasks do not depend on each other.

Example:

Get Delhi weather

Get Mumbai weather

These can potentially run independently.



This can enable parallel execution.

20. Replanning



Sometimes the original plan stops being useful.

Example:

Plan:

1. Search internal database
2. Analyze result
3. Generate answer

But:

Step 1 Result:

No information found.

The agent may need to modify the plan:

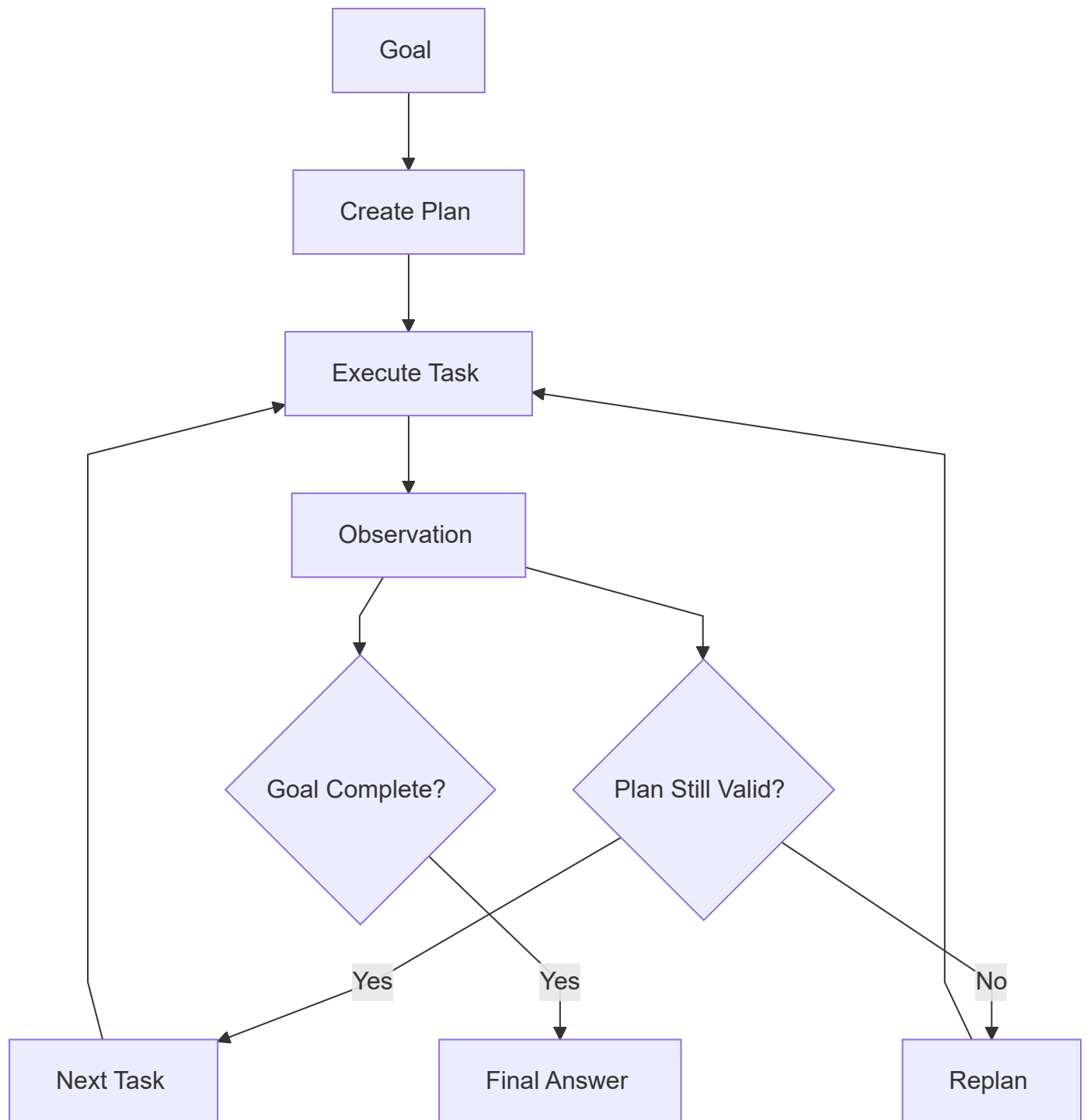
New Plan:

1. Search external source
2. Analyze result
3. Generate answer

This is:

Replanning

21. Replanning Architecture



22. When Should an Agent Replan?

Possible situations:

- Tool failed
- Required information unavailable
- Unexpected result
- Original assumption was wrong
- New information changes the task
- User changes requirements

Replanning gives the agent flexibility.

23. Replanning Should Be Controlled

Do not allow:

```
Plan
↓
Fail
↓
Replan
↓
Fail
↓
Replan
↓
Fail
↓
Replan forever
```

Use limits such as:

```
MAX_REPLANS = 2
```

Then:

```
Replan Limit Reached
↓
Fallback / Ask User / Stop
```

24. What is Reflection?



Reflection is a pattern where an agent evaluates an intermediate result or output and decides whether it should be accepted, improved, corrected or retried.

Basic flow:

```
graph TD
    A[Generate / Execute] --> B[Evaluate Result]
    B --> C{Good Enough?}
    C -- ✓ --> D[Yes]
    C -- ✗ --> E[No]
    D --> F[Continue]
    E --> G[Improve]
    G --> H[Retry]
```

25. Reflection Example

Goal:

```
Find information about a company.
```

Agent searches:

```
search_web("ABC")
```

Observation:

```
Results about ABC television network.
```

But the user meant:

```
ABC Software Pvt Ltd.
```

Agent evaluates the result:

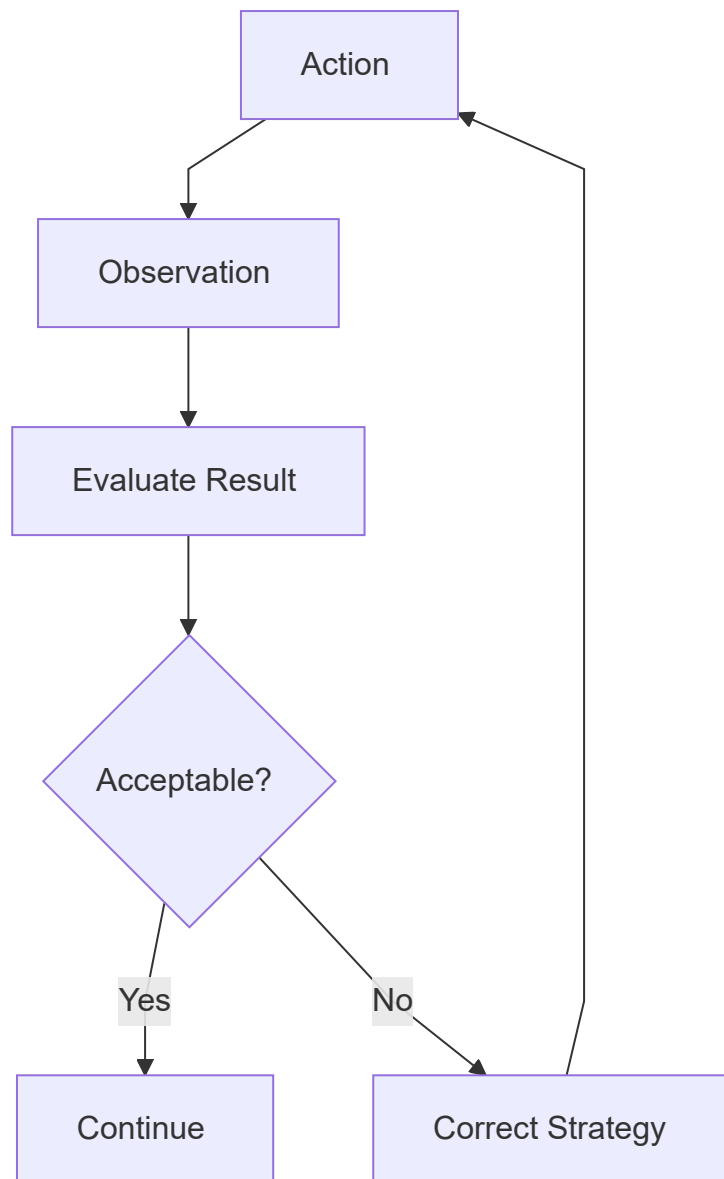
This does not appear relevant.

Then:

Search again with:
"ABC Software Pvt Ltd"

This is a simple reflection/correction loop.

26. Reflection Architecture



27. Reflection vs Replanning

They are related but slightly different.

Reflection

was this result/output good enough?

Example:

Search result is irrelevant.

Replanning

Does the overall plan need to change?

Example:

Internal search cannot solve the task,
so use another information source.

Simple distinction:

Reflection

→ Evaluate result

Replanning

→ Change plan

28. Reflection Does NOT Guarantee Correctness



An LLM evaluating its own result can still be wrong.

Therefore:

LLM says:
"This answer is correct."

does not guarantee correctness.

For important tasks, combine reflection with:

- Deterministic validation
- External verification
- Schema validation
- Tests
- Human review
- Independent evaluator

29. Retry

A retry means attempting an operation again after failure.

Example:

```
Weather API
↓
Timeout
↓
Retry
↓
Success
```

Basic implementation:

```
MAX_RETRIES = 2

for attempt in range(MAX_RETRIES):

    try:

        result = call_tool()

        break

    except TimeoutError:

        continue
```

30. Retry vs Reflection vs Replanning



Understand this distinction:

RETRY

"Try the same operation again."

Example:

API timeout

→ Call same API again

REFLECTION

"Was the result good enough?"

Example:

Search result irrelevant

→ Improve query

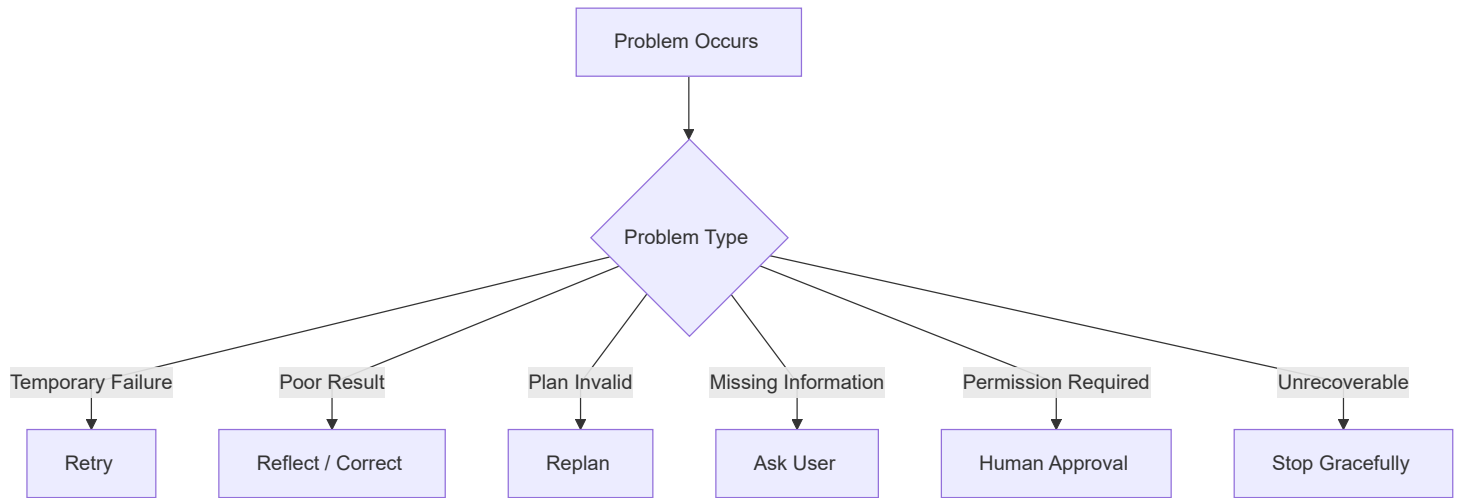
REPLANNING

"Should I change the overall approach?"

Example:

Internal search failed
→ Switch to another source

31. Error Recovery Decision



This is a useful practical mental model.

32. Missing Information

Sometimes the correct action is not another tool call.

Example:

User:
check my order status.

But no order is identified.

Bad agent:

Guess an order ID.

Correct:

Ask User:
which order would you like me to check?

Agents should know when information is missing.

33. Asking User vs Guessing



A reliable agent should not invent critical parameters.

Example:

`send_email(recipient, message)`

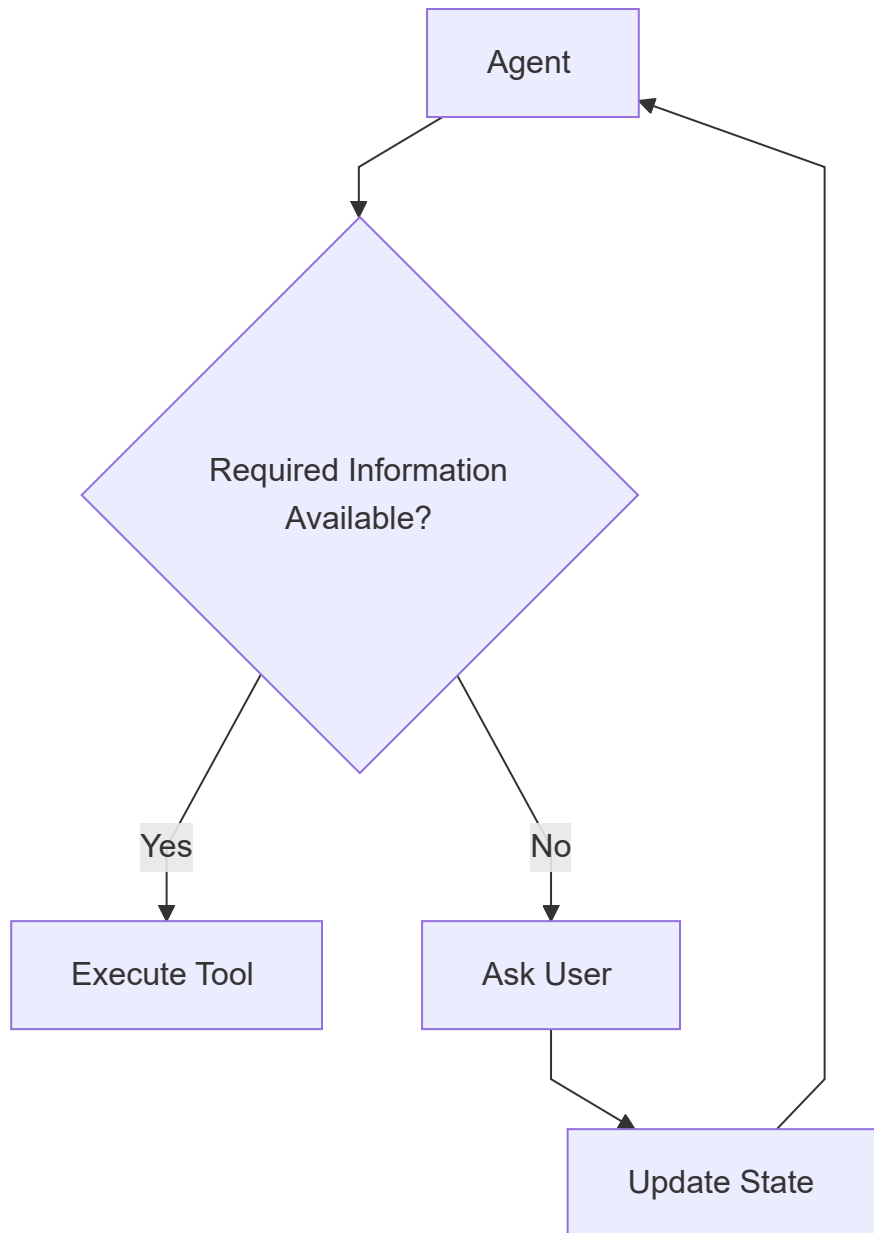
If recipient is missing:

Ask User

not:

Guess Recipient

Flow:



34. Human-in-the-Loop During Planning

Some plans may include sensitive actions.

Example:

Goal:

Prepare and send a report.

Plan:

1. Collect information

2. Generate report

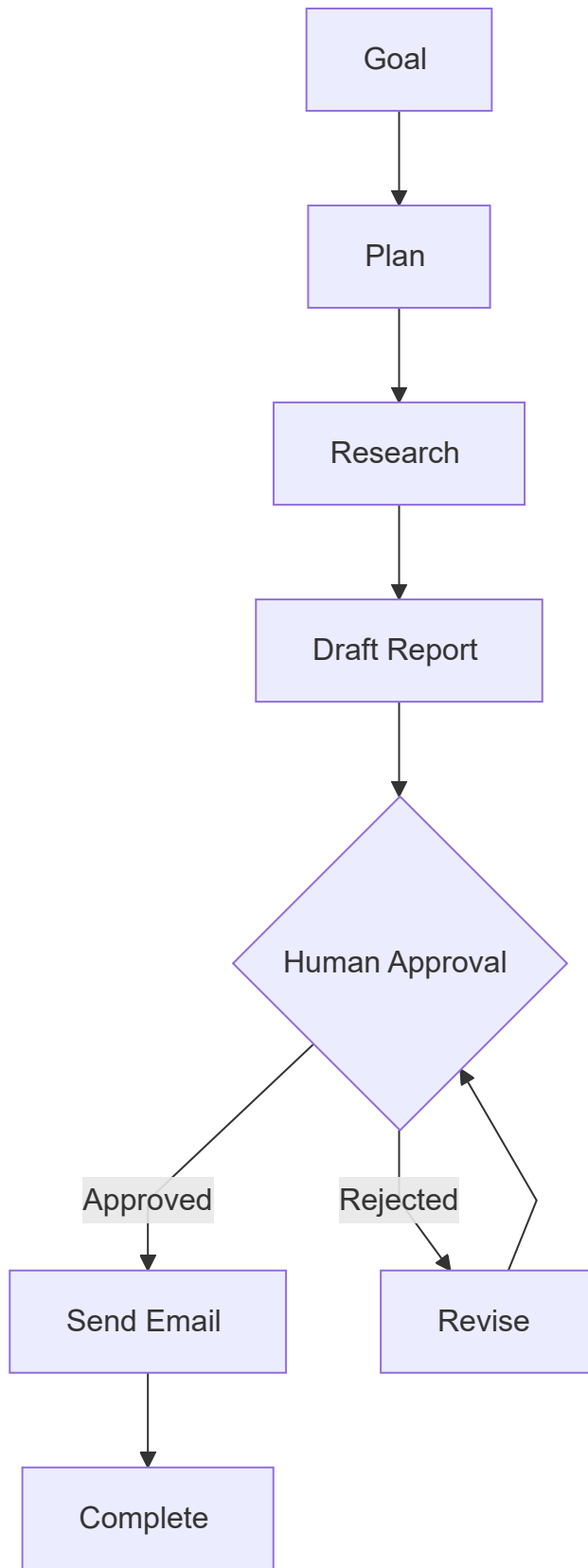
3. Review report

4. Send email

Before step 4:

Human Approval

Architecture:



35. Planning Should Not Control Security



Suppose an agent plans:

- 1. Find user
- 2. Delete account

*The fact that the LLM created this plan does **not** mean the action is authorized.*

Backend must still enforce:

- Authentication
- Authorization
- Permissions
- Business Rules
- Human Approval

Remember:

- Agent Plan
 - ≠
 - Permission
-

36. Planning + Deterministic Rules

Good architecture:

LLM

→ Flexible planning

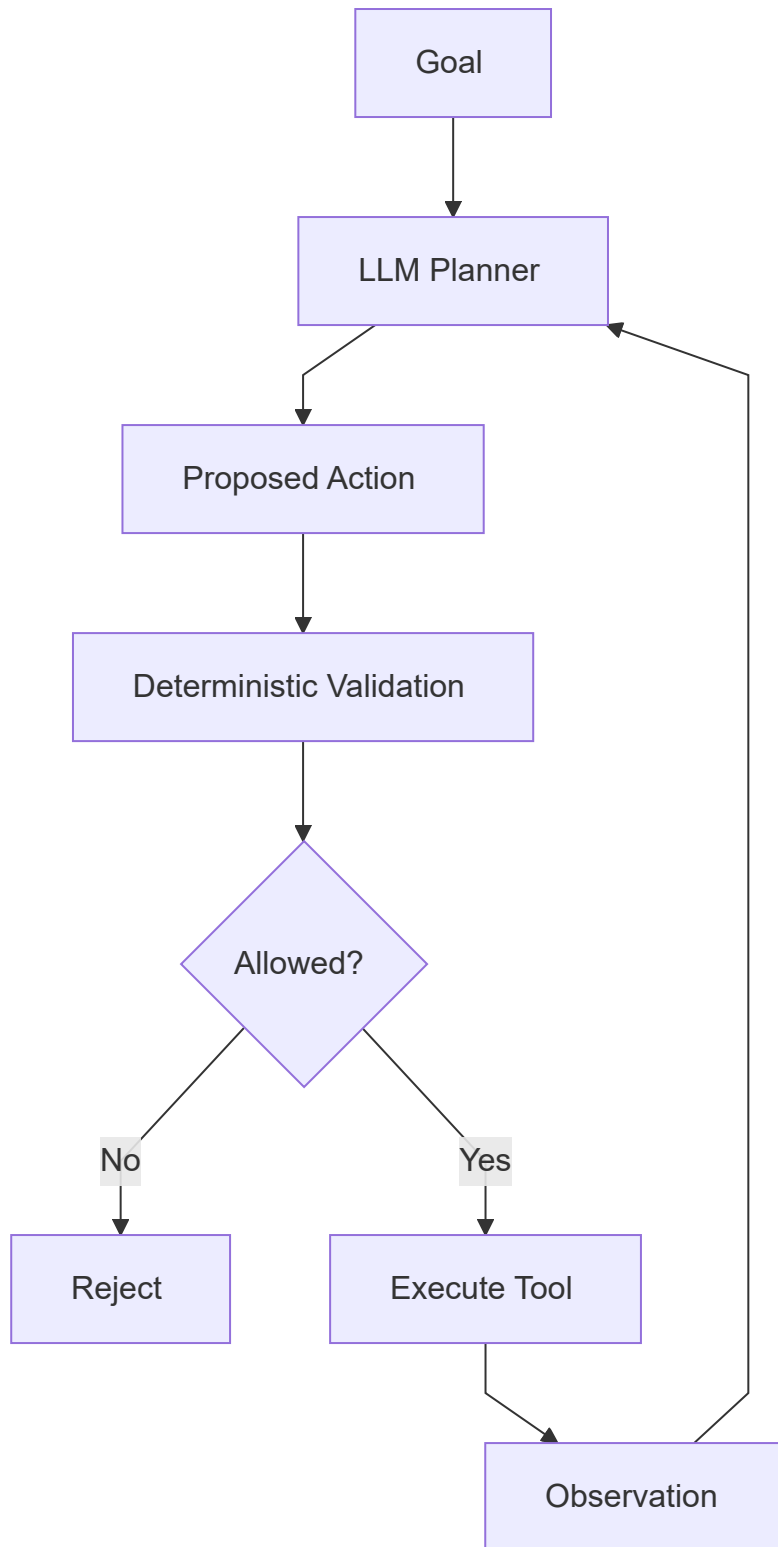
Application

→ Workflow control

Backend

→ Security and business rules

Example:



37. Simple Planner Implementation

Let's implement a basic plan representation.

```
def create_plan():  
  
    return [  
        "get_delhi_weather",  
        "get_mumbai_weather",  
        "compare_weather"  
    ]
```

State:

```
state = {  
  
    "plan":  
        create_plan(),  
  
    "current_step":  
        0,  
  
    "results":  
        {}  
}
```

38. Executing the Plan

```
def execute_step(state):  
  
    step = state["plan"][  
        state["current_step"]  
    ]  
  
    if step == "get_delhi_weather":  
  
        state["results"]["delhi"] = (  
            get_weather("Delhi")
```

```

    )

elif step == "get_mumbai_weather":

    state["results"]["mumbai"] = (
        get_weather("Mumbai")
    )

elif step == "compare_weather":

    state["results"]["comparison"] = (
        compare_weather(
            state["results"]["delhi"],
            state["results"]["mumbai"]
        )
    )

state["current_step"] += 1

```

39. Plan Execution Loop

```

while (
    state["current_step"]
    < len(state["plan"])
):

    execute_step(state)

```

This is a fixed plan.

An LLM-based planner would dynamically generate or modify the plan instead.

40. LLM-Based Planning Concept

Conceptually:

```
plan = llm.invoke(
    """
    Goal:
    Compare Delhi and Mumbai weather.

    Create a short structured plan
    using the available tools.
    """
)
```

Possible structured output:

```
{
  "steps": [
    {
      "tool": "get_weather",
      "arguments": {
        "city": "Delhi"
      }
    },
    {
      "tool": "get_weather",
      "arguments": {
        "city": "Mumbai"
      }
    },
    {
      "action":
        "compare_results"
    }
  ]
}
```

```
}
```

In production, structured output is preferable to parsing arbitrary free-form text.

41. Structured Plans



Prefer:

```
{
  "steps": [
    {
      "id": 1,
      "action": "get_weather",
      "city": "Delhi"
    },
    {
      "id": 2,
      "action": "get_weather",
      "city": "Mumbai"
    }
  ]
}
```

over:

First maybe search Delhi and then perhaps
Mumbai and after that compare...

Structured plans are easier to:

Validate

Execute

Store

Track

Retry

42. Planning State

A more realistic planning state could contain:

```
state = {  
  
  "goal":  
    "Compare Delhi and Mumbai",  
  
  "plan": [],  
  
  "current_step": 0,  
  
  "completed_steps": [],  
  
  "results": {},  
  
  "errors": [],  
  
  "replans": 0,  
  
  "completed": False  
}
```

This state evolves throughout the workflow.

43. Planning + Checkpointing

Suppose:

10-step workflow

After step 6:

Application crashes.

Without checkpoint:

Restart from Step 1

With checkpoint:

Load State
↓
current_step = 6
↓
Continue

Architecture:



This is one reason state and planning are closely connected.

44. Planning + Memory

Memory may influence planning.

Example memory:

User prefers low-cost travel options.

Goal:

Plan a trip.

The planner can use:

Goal
+
Relevant Memory
+
Available Tools

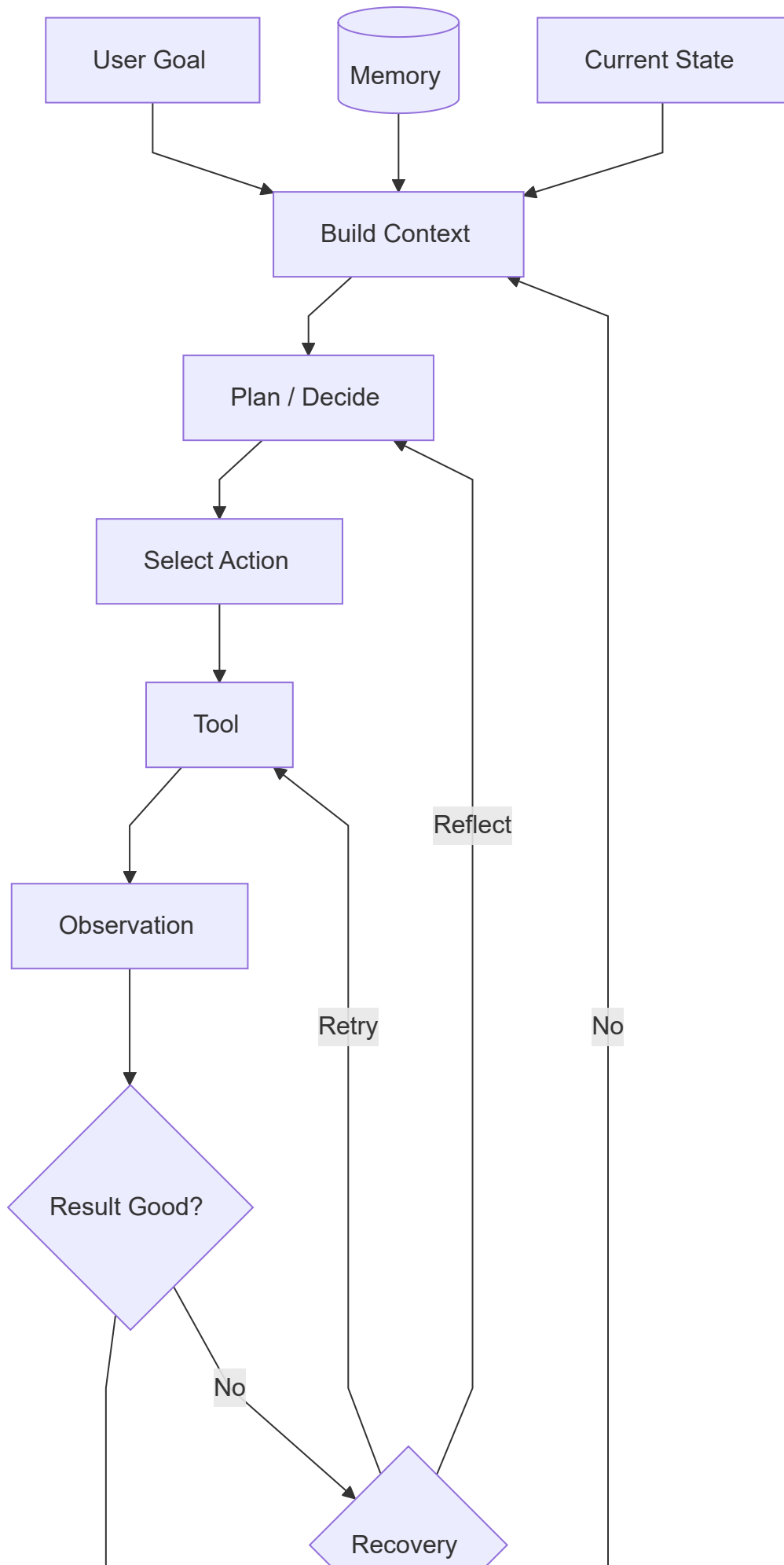
to create a plan focused on:

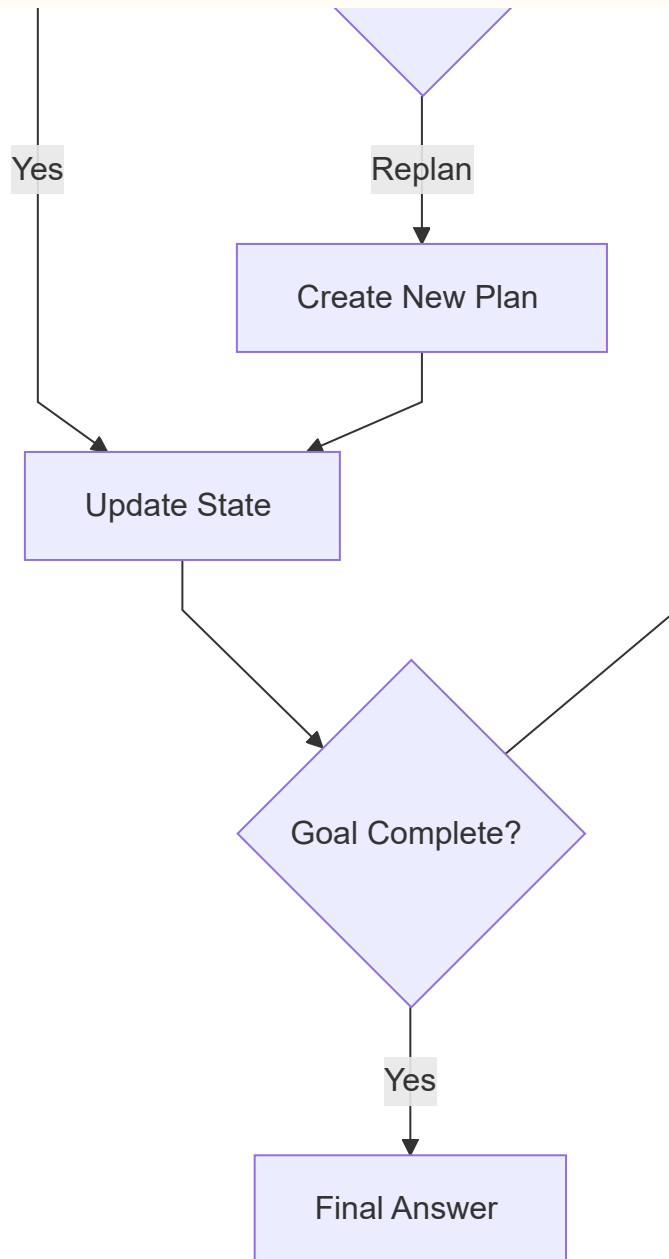
Budget-friendly options

But only relevant memories should be included.

45. Planning + Tools + State + Memory

We can now combine everything from Chapters 1-3.





This is now a proper agentic workflow.

46. Do All Agents Need Explicit Planning?



No.

For simple tasks:



```
graph TD; User --> Agent; Agent --> Tool; Tool --> Answer;
```

User
↓
Agent
↓
Tool
↓
Answer

an explicit plan may add unnecessary complexity.

For more complex tasks:



```
graph TD; Goal --> Steps[Multiple dependent steps]; Steps --> Conditions[Changing conditions]; Conditions --> Tools[Different tools];
```

Goal
↓
Multiple dependent steps
↓
Changing conditions
↓
Different tools

planning becomes more useful.

Therefore:



```
graph TD; SimpleTask[Simple Task] --> ReactiveAgent[→ Reactive Agent]; ComplexTask[Complex Task] --> Planning[→ Planning may help];
```

Simple Task
→ Reactive Agent

Complex Task
→ Planning may help

47. Do All Agents Need Reflection?

No.

Reflection adds:

Extra LLM calls

Latency

Cost

Complexity

Use it when evaluating intermediate results genuinely improves reliability.

Do not add:

Planning

+

Reflection

+

Replanning

+

Critic Agent

+

Supervisor

to every simple application.

48. Agent Complexity Principle



Always prefer:

Simplest architecture
that reliably solves the task.

Progression:

Normal Code
↓ if insufficient

LLM
↓ if external capability needed

LLM + Tools
↓ if multi-step dynamic execution needed

Agent Loop
↓ if explicit orchestration needed

LangGraph
↓ if workflow becomes complex

Multi-Agent
↓ only if specialization genuinely helps

This is an important engineering mindset.

49. Common Planning Mistakes

Mistake 1 – Planning Every Simple Task

Not every query needs a plan.

Mistake 2 – Huge Plans

Prefer manageable plans that can adapt.

Mistake 3 – No Replanning Limit

Can create endless loops.

Mistake 4 – Confusing Retry With Replanning

Remember:

Retry

→ Same operation again

Replan

→ Change strategy

Mistake 5 – Trusting Self-Reflection Completely

LLMs can incorrectly evaluate their own outputs.

Mistake 6 – Guessing Missing Information

Ask the user when important information is required.

Mistake 7 – Treating Plan as Authorization

Backend security rules always remain authoritative.

50. Important Interview Questions

Q1. What is planning in an AI agent?



Answer:

Planning is the process of determining the sequence of tasks or actions required to achieve a goal, potentially adapting the plan based on observations during execution.

Q2. What is task decomposition?



Answer:

Task decomposition is the process of breaking a complex goal into smaller manageable tasks that can be executed individually.

Q3. What is the Plan-and-Execute pattern?



Answer:

In the Plan-and-Execute pattern, a planner first creates a set of tasks for achieving the goal, and an executor then performs those tasks using available tools and observations.

Q4. Planner vs executor?

Answer:

The planner determines what needs to be done and in what order, while the executor performs the selected tasks or tool calls.

Q5. What is reactive planning?

Answer:

Reactive planning means deciding the next action based on the current state and latest observation instead of generating a complete plan in advance.

Q6. What is replanning? ★ ★ ★ ★ ★

Answer:

Replanning is the process of modifying or replacing the existing plan when new observations, failures or changed conditions make the original plan unsuitable.

Q7. What is reflection in an AI agent?



Answer:

Reflection is a pattern where the agent evaluates an intermediate result or output and decides whether it should be accepted, corrected or retried.

Q8. Retry vs reflection vs replanning?



Answer:

A retry repeats an operation after failure. Reflection evaluates whether a result is satisfactory and may correct it. Replanning changes the broader strategy or sequence of actions.

Q9. Does reflection guarantee correctness?

Answer:

No. LLMs can incorrectly evaluate their own outputs, so important systems should combine reflection with deterministic validation, external verification or human review.

Q10. Why use structured plans?

Answer:

Structured plans are easier to validate, execute, store, track and modify programmatically than free-form natural-language plans.

Q11. Do all agents need planning?



Answer:

No. Simple agents can reactively choose the next action. Explicit planning is more useful for complex tasks involving multiple dependent steps or changing conditions.

Q12. Why limit retries and replanning?

Answer:

Limits prevent infinite loops, excessive latency, unnecessary tool calls and unexpected costs.

Q13. What should an agent do when critical information is missing?

Answer:

The agent should request the missing information from the user rather than inventing critical values or making unsafe assumptions.

Q14. Can an agent plan a restricted action and execute it automatically?

Answer:

No. A model-generated plan does not provide authorization. Authentication, authorization, business rules and approval requirements must still be enforced by the application.

51. Final Mental Model

Remember the four important concepts:

PLANNING

"What steps should achieve the goal?"

TASK DECOMPOSITION

"How can I break the goal into smaller tasks?"

REFLECTION

"Was the result good enough?"

REPLANNING

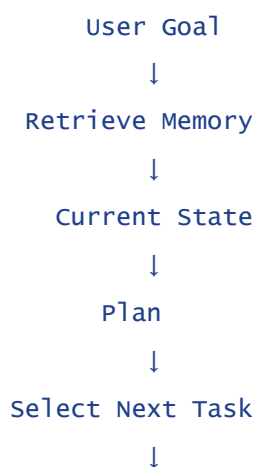
"Does my strategy need to change?"

And:

RETRY

"Should I try the same operation again?"

Complete flow:





At this point, the core agent foundation is:

LLM
+
Tools
+
State
+
Memory
+
Planning
+
Control Loop

The next major step is to stop managing all these loops, states and branches manually and represent them as an explicit graph:

State



Node



Edge



Node



Conditional Edge



Tool



Loop

That is exactly what:

LangGraph

is designed to help us build.

End of Chapter 3 – State, Memory & Planning
